Field Hours

CSCE 726 Large-Scale Optimization for Machine Learning

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Gradients and convex objectives

An objective is convex when the segment between any two points lies on or above the graph. For f(x) = 0.5 x squared, the second derivative is 1, so it is convex, and the gradient x is zero only at x = 0. You use the gradient as a local linear model. At x = 3 the gradient is 3, and a step of size 0.1 moves you to 2.7. The value falls from 4.5 to 3.645.

A training loss is usually an average of per-example terms plus a regularizer. With n = 1000 the full gradient averages 1000 individual gradients. Convexity of each piece passes to the average, so squared loss or logistic loss on a linear model stays convex in the weights. A network with one hidden layer of 32 units is not convex in its weights. You still compute a gradient, and you do not inherit a global-minimum guarantee.

What the training objective adds up.

Loss on one example

Average over the training set

Regularizer on the weights

Gradient of that total

One step on a quadratic

1. f(x) = 0.5 x squared. At x = 3 the value is 4.5 and the gradient is 3.

2. A step of size 0.1 lands at 2.7. The new value is 0.5 times 2.7 squared, so 3.645. The decrease is 4.5 minus 3.645, so 0.855.

3. A step of size 2 from 3 lands at 3 minus 6, so -3, and the value is still 4.5. A step that is too long need not descend, even on this convex function.

Problem 1. After the step of size 0.1 from x = 3, what are the new x and the new value?

Answer. x is 2.7 and the value is 3.645.

Problem 2. Why does convex logistic regression not give you the same global claim for a hidden layer of 32 units?

Answer. That hidden layer is not a convex function of its weights. A point where the gradient is zero can be a saddle or a poor local minimum. Convexity was what made every stationary point global, and this model does not have it.

Lecture 2 · 90 minutes

Step sizes

You repeat one move in gradient descent: subtract the step size times the gradient. On f(x) = 0.5 L x squared the gradient is L x, so the error multiplies by 1 minus eta times L each step. If L is 4 and eta is 0.1, the factor is 0.6, and an error of 10 shrinks to 3.6 after 2 steps. If eta exceeds 2 divided by L, here 0.5, the absolute factor exceeds 1 and the iterates diverge.

You rarely know L. A backtracking search tries a step and shrinks eta by a factor of 0.5 until the value falls in line with the linear model. On a training loss you may instead try a grid such as 0.001, 0.01, and 0.1, and keep a value whose curve descends. A curve that climbs by a factor of 10 in the first 5 steps is a step size that is too large, not a model that cannot fit.

Propose a step, then accept or shrink.

1. Evaluate the gradient

2. Propose the new point

3. Accept if the value falls

4. Otherwise shrink the step

A known curvature

1. L is 4 and eta is 0.1, so each step multiplies x by 0.6. Start at x = 10.

2. After one step x is 6. After two steps x is 3.6.

3. Here f(x) = 2 x squared, because 0.5 times L is 2. Then f(10) is 200 and f(3.6) is 25.92. The contraction factor on x is 0.6, which you can check before you look at the plot.

Problem 1. With L = 4 and eta = 0.1, what is x after 2 steps from 10, and what is the factor per step?

Answer. x is 3.6. The factor is 0.6, which is 1 minus 0.1 times 4.

Problem 2. You set eta = 1 with L = 4. What is the factor, and do the iterates converge?

Answer. The factor is 1 minus 4, so -3. Its absolute value is 3, which is greater than 1, so the iterates diverge. The lecture's upper limit is 2 divided by L, which is 0.5.

Lecture 3 · 90 minutes

Stochastic gradients

A stochastic gradient replaces the average over all n examples by the gradient of one example, or of a minibatch. With n = 10000, one full gradient costs about 10000 example gradients. A minibatch of 32 cuts the variance of the estimate relative to a single example by about a factor of 32 when the example gradients are uncorrelated, because the variance of an average scales as 1 over the batch size. You still take many steps per pass.

If you keep the step size constant, the noise does not vanish and the iterate jitters near a minimizer. A schedule of 0.1 divided by the step index t drives that noise down. At step 1 the step size is 0.1. At step 100 it is 0.001. You plot loss against examples seen. Otherwise a batch of 32 and a batch of 256 look like the same effort when they

are not.

One stochastic step.

Draw 32 examples -> Average their gradients -> Step with the current eta -> Count examples seen

One noisy step versus the full gradient

1. The weight is 2. The full gradient is 6. One example draw gives gradient 10. The step size is 0.05.

2. The stochastic update lands at 2 minus 0.05 times 10, so 1.5.

3. The full-gradient update would land at 2 minus 0.05 times 6, so 1.7. The extra 0.2 of motion is the example gradient sitting 4 above the mean, times 0.05.

Problem 1. What weight do you get after the stochastic step, and what would the full gradient have produced?

Answer. The stochastic step lands at 1.5. The full gradient would land at 1.7.

Problem 2. Example gradients are uncorrelated. How does the variance of a batch-32 average compare with a single example?

Answer. It is about 1/32 of the single-example variance. You pay 32 examples per step for that reduction.

Lecture 4 · 90 minutes

Momentum

Momentum keeps a velocity. You set it to beta times its old value plus the new gradient, then subtract the step size times that velocity. With beta 0.9, step size 0.1, and a constant gradient of 2, velocity approaches 2 divided by 0.1, so 20. The effective step approaches 2, which is 10 times the plain step of 0.2. That factor is why you shrink the step size when you turn momentum on.

You see the plain gradient zigzag on a long narrow valley, because the steep walls dominate each step. The velocity averages those wall components and keeps the component along the floor. With beta 0.9, a gradient from 10 steps ago still has weight 0.9 to the tenth power, about 0.35, so recent gradients lead and older ones still contribute. Nesterov momentum evaluates the gradient at a looked-ahead point instead.

One heavy-ball update.

1. Decay the old velocity

2. Add the new gradient

3. Move by eta times velocity

Two steps from rest

1. Start velocity at 0 and x at 0. The gradient is the constant 2. Beta is 0.9 and the step size is 0.1.

2. After one step the velocity is 2 and x is -0.2.

3. After the second step the velocity is 0.9 times 2 plus 2, so 3.8, and x is -0.2 minus 0.1 times 3.8, so

-0.58. Equilibrium velocity is 20, so you are still building speed.

Problem 1. What are the velocity and x after the two steps in the example?

Answer. Velocity is 3.8 and x is -0.58.

Problem 2. Why do you often lower the step size when you set beta to 0.9?

Answer. At equilibrium the velocity is the gradient divided by 1 minus 0.9, a factor of 10. The same step size then moves you 10 times farther than plain gradient descent. A step size that was stable without momentum can overshoot once that factor appears.

Lecture 5 · 90 minutes

Adaptive steps

You let AdaGrad divide each coordinate's step by the square root of the sum of that coordinate's past squared gradients, plus a tiny constant such as 1e-8. A coordinate whose gradients stay near 0.01 keeps a larger step than a coordinate whose gradients are near 10. The sum never forgets. After 10000 similar steps the denominator has grown like the square root of 10000, which is 100, so the steps shrink even if the curvature did not change.

RMSProp replaces the sum by an exponential average with decay 0.99, so old squared gradients fade. Adam also averages the gradient with decay 0.9, and corrects bias because both averages start at 0. At step 1 the corrected momentum equals the first gradient. You still choose a base step, often 0.001. Adam does not choose it for you. A base step of 0.1 can diverge where 0.001 was stable.

Two per-coordinate scalings.

AdaGrad

Sum every squared gradient

Denominator grows like sqrt(t)

Per-coordinate steps

Adam

Exponential average of squares

Momentum with bias correction

A base step you still set

Two AdaGrad steps

1. One coordinate starts at 0. Both gradients are 2. Ignore the tiny constant. The base step is 0.5.

2. After the first step the sum of squares is 4 and the denominator is 2. You subtract 0.5 times 2 divided by 2, so 0.5. The coordinate is -0.5.

3. After the second step the sum of squares is 8 and the denominator is the square root of 8, about 2.828. You subtract 1 divided by that square root, about 0.354. The coordinate is about -0.854.

Problem 1. To three decimals, where is the coordinate after the two AdaGrad steps?

Answer. -0.854. The first step subtracts 0.5. The second subtracts 1 over the square root of 8, about 0.354.

Problem 2. Why can AdaGrad's step vanish late in a run where RMSProp would keep moving?

Answer. AdaGrad's denominator is the square root of a sum that keeps every past squared gradient, so it grows like the square root of the step count. RMSProp uses an exponential average, for example with decay 0.99, which forgets old gradients and can stay at a finite scale.

Lecture 6 · 90 minutes

What a rate actually says

For a smooth convex function, gradient descent with a fixed step drives the error down like 1 over k after k steps. Stochastic gradient on a convex Lipschitz objective, with a decaying step, has worst-case error like 1 over the square root of k. After 100 steps that bound has improved by a factor of 10, because the square root of 100 is 10. The bound is a guarantee, not a prediction of your plot.

A deep network's training loss is not convex. The same algorithms still run, and a curve can fall faster than the bound or stall with gradient norm under 0.001 while training error is still 0.4. You report the quantity the guarantee names. One tiny gradient at step 100 does not finish a claim about an average of past norms. Training loss 0.01 beside validation loss 0.4 is memorization, and a smaller step will not close that gap.

A bound is not the same object as a curve.

Convex guarantee

Error like 1 over sqrt(k)

A decaying step size

A worst-case statement

One training plot

A nonconvex loss curve

A saddle with a tiny gradient

Validation next to training

Instantiate one worst-case expression

1. Take the illustration bound 5 divided by the square root of k. At k = 1 it equals 5.

2. At k = 100 it equals 0.5. At k = 10000 it equals 0.05.

3. A training curve that reads 2.0, then 0.8, then 0.3 by step 100 beat this illustration. That is allowed. A bound is not a tight description of one easy problem. A gradient norm of 0.0005 with loss still at 0.4 only says the first-order test is small.

Problem 1. Using 5 divided by the square root of k, what is the bound at k = 100 and at k = 10000?

Answer. 0.5 and 0.05.

Problem 2. Training loss is 0.01 and validation loss is 0.4. What does a theorem about training gradient norms fail to tell you?

Answer. Whether the model predicts new points. The theorem and a tiny training gradient speak about the training objective. The gap of 0.39 is a generalization failure, and shrinking the step size does not correct it.

Lecture 7 · 90 minutes

Checks, decay, and schedules

Before a long run you check one backpropagated coordinate against a finite difference. Move that scalar weight by 1e-4 and divide the change in loss by 1e-4. If the loss rises by 2e-4, the quotient is 2. A backpropagated value of 2.0 agrees. A backpropagated value of 0.2 means the graph is wrong on that coordinate. No optimizer repairs a gradient that points the wrong way.

Weight decay adds (lambda over 2) times the squared weight length, with lambda such as 1e-4. The extra gradient is lambda times the weight. At weight 3 that pull is 0.0003, small beside a data gradient of size 1 and large beside one of size 0.0001. Multiplying the step size by 0.1 after 5 stalled epochs is a second knob. You change one knob at a time, or the plot cannot tell you which change mattered.

Check one coordinate before the long run.

1. Record the loss

2. Perturb one weight by 1e-4

3. Divide the loss change by 1e-4

4. Compare with backpropagation

A check, then the decay term

1. The loss rises by 2e-4 when the weight rises by 1e-4. The quotient is 2. Backpropagation says 2. This coordinate passes.

2. Separately, lambda is 1e-4 and the weight is 3. The extra gradient is 0.0003. The decay's own contribution to the loss is (1e-4 divided by 2) times 9, so 4.5e-4.

3. If you add lambda divided by 2 times the weight, you add 0.00015 and you have implemented a different regularizer than the one you wrote down. Log the data term and the decay term separately.

Problem 1. The loss rises by 2e-4 when one weight rises by 1e-4. What quotient do you compare with backpropagation?

Answer. 2.

Problem 2. Lambda is 1e-4 and the weight is 3. What extra gradient does weight decay add, and what does it add to the loss value?

Answer. The extra gradient is 0.0003. The loss contribution is 4.5e-4.

Lecture 8 · 90 minutes

The plot you owe

You can now take a gradient step, choose a step size that contracts a quadratic of curvature L, and replace the full gradient by a minibatch of 32. You can add momentum with a coefficient of 0.9, divide coordinates the way AdaGrad or Adam does, and state a 1 over square root of k bound when its assumptions hold. You can also check one backpropagated coordinate against a finite difference of size 1e-4.

You can implement SGD, momentum, and Adam on one model and plot the loss against examples seen. That plot is the assignment. If momentum with beta 0.9 and the same step size diverges while SGD descends, the factor 10 is the first place you look. If Adam descends at base step 0.001 and diverges at 0.1, you have measured the base step the method did not choose for you.

The run you can now finish.

Check one finite difference -> Train SGD, momentum, and Adam -> Plot loss against examples seen -> Read divergence off the step

Momentum on a noisy gradient

1. Plain SGD uses step size 0.01 on a batch gradient estimate of 4 from 32 examples. The step length is 0.04.

2. Momentum with beta 0.9, once the velocity has reached equilibrium, multiplies that effective step by 10. The same step size and the same gradient estimate move you by 0.4.

3. If that momentum curve oscillates, you cut the step size by about 10, down to 0.001, before you blame the minibatch noise. The finite-difference check should already have passed, or you are stepping on a bad gradient.

Problem 1. Combine the momentum equilibrium with a stochastic step. The step size is 0.01, the batch gradient estimate is 4, and beta is 0.9 at equilibrium. How long is the effective step, and what step size restores a length of 0.04?

Answer. The effective step is 0.4, because equilibrium multiplies by 10. Restoring a length of 0.04 means dividing the step size by 10, so the step size is 0.001.

Problem 2. A finite-difference quotient on one weight is 2, and backpropagation says 0.2. Adam then diverges. What do you fix first?

Answer. The gradient. The quotient 2 and the backpropagated 0.2 disagree by a factor of 10, so that coordinate is wrong. Changing Adam's base step does not repair a graph that points the wrong way.